

Quiz App - Spring Boot + Spring Security + JWT

A RESTful Quiz Application built with Spring Boot, secured with JWT Authentication and Spring Security.

🔧 Tech Stack

- **Backend:** Java, Spring Boot
 - **Security:** Spring Security, JWT (JSON Web Token)
 - **Database:** PostgreSQL
 - **ORM:** Spring Data JPA / Hibernate
 - **Build Tool:** Maven
 - **API Docs:** Swagger UI
-

🚀 Features

- User Registration & Login with JWT Authentication
 - BCrypt Password Hashing
 - Stateless Session Management
 - Create quizzes by category and difficulty level
 - Random question selection using native SQL query
 - Submit quiz and get score as percentage
 - DTO pattern to hide sensitive data (correct answers) from client
-

🔒 Authentication Flow

1. Register → `POST /register`
 2. Login → `POST /login` → returns JWT token
 3. Use token in `Authorization: Bearer <token>` header for all protected endpoints
-

Project Structure

```
src/main/java/com/quizapp/quizapplication/  
├── config/  
│   ├── JwtFilter.java  
│   └── SecurityConfiguration.java  
├── controller/  
│   ├── QuestionController.java  
│   ├── QuizController.java  
│   └── UserController.java  
├── model/  
│   ├── Question.java  
│   ├── QuestionDTO.java  
│   ├── Quiz.java  
│   ├── Response.java  
│   ├── Users.java  
│   └── UserPrinciples.java  
├── repository/  
│   ├── QuestionRepo.java  
│   ├── QuizRepo.java  
│   └── UserRepo.java  
└── service/  
    ├── JWTservice.java  
    ├── QuestionService.java  
    ├── QuizService.java  
    ├── UserDetailsService.java  
    └── UserService.java
```

API Endpoints

Auth

Method	Endpoint	Description	Auth Required
POST	<code>/register</code>	Register new user	No
POST	<code>/login</code>	Login and get JWT token	No

Questions

Method	Endpoint	Description	Auth Required
GET	<code>/question/allQuestion</code>	Get all questions	Yes
GET	<code>/question/difficulty/{level}</code>	Get questions by difficulty	Yes
POST	<code>/question/add</code>	Add a new question	Yes

Quiz

Method	Endpoint	Description	Auth Required
POST	<code>/quiz/create</code>	Create a quiz	Yes
GET	<code>/quiz/get</code>	Get quiz questions (DTO)	Yes
POST	<code>/quiz/submit/{quizId}</code>	Submit answers and get score	Yes

Setup & Run

Prerequisites

- Java 17+
- PostgreSQL
- Maven

Steps

1. Clone the repo

```
bash
git clone https://github.com/Amanverma17/quiz-app-spring-security.git
cd quiz-app-spring-security
```

2. Configure database in `src/main/resources/application.properties`

```
properties
```

```
spring.datasource.url=jdbc:postgresql://localhost:5432/your_db
spring.datasource.username=your_username
spring.datasource.password=your_password
spring.jpa.hibernate.ddl-auto=update
```

3. Run the app

```
bash

./mvnw spring-boot:run
```

4. Access Swagger UI

```
http://localhost:8080/swagger-ui/index.html
```

Key Concepts Implemented

- JWT Filter (`OncePerRequestFilter`) for stateless auth
- `DaoAuthenticationProvider` with BCrypt password encoding
- Custom `UserDetailsService` and `UserPrinciples`
- `ManyToMany` relationship between Quiz and Question
- Native `@Query` with `RANDOM()` for random question selection
- DTO pattern to prevent exposing correct answers to client
- `ResponseEntity` with proper HTTP status codes

Author

Aman Verma

[GitHub](#)